



Universidade do Minho

UNIVERSIDADE DO MINHO
MESTRADO EM CIBERSEGURANÇA
2025/2026

VPN Lab: The Container Version

PG59783 - Carlos Daniel Silva Fernandes - Mestrado em Cibersegurança
PG59790 - Luís Filipe Pinheiro Silva - Mestrado em Cibersegurança
PG59791 - Pedro Augusto Ennes de Martino Camargo - Mestrado em
Cibersegurança

Ao longo do trabalho prático **VPN Lab: The Container Version** foram desenvolvidos e executados vários scripts que implementam as técnicas e os testes propostos no guião. Cada script encontra-se devidamente identificado.

Questão 2.a

Após a execução do script `tun.py`, é criada uma interface virtual denominada `tun`. Através do comando `ip address`, é possível verificar a sua existência e respetivas configurações.

```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN group default qlen 500
    link/none
```

Figura 1: Interface `tun`

Posteriormente, ao alterar o código do script, é possível criar uma interface com um nome personalizado. Neste caso, foi criada a interface `ferna`, correspondente às primeiras cinco letras do apelido *Fernandes*.

```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
4: ferna0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN group default qlen 500
    link/none
```

Figura 2: Interface `ferna`

Questão 2.b

Antes de podermos utilizar as interfaces, é necessário configurá-las devidamente. Para tal, deve ser atribuído um endereço IP e a interface deve ser

ativada. Essas operações são realizadas através dos seguintes comandos:

```
ip addr add 192.168.53.99/24 dev tun0
ip link set dev tun0 up
```

Após a execução destes comandos, a interface torna-se totalmente funcional. Podemos verificar a sua configuração utilizando o comando `ip address`, conforme ilustrado na Figura 3.

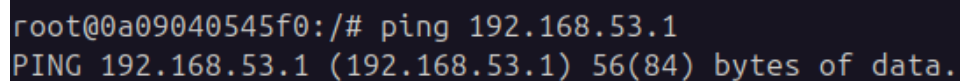


```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
6: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 192.168.53.99/24 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::8260:61d5:78e3:4cd/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
```

Figura 3: Interface `tun` após a configuração.

Questão 2.c

Nesta tarefa, procedeu-se à análise do tráfego direcionado para a nossa interface, criando para isso um objeto IP no *Scapy*. Ao executarmos um `ping` para o endereço `192.168.53.1`, é possível observar que a interface está a reencaminhar corretamente os pacotes. Isto acontece porque o endereço IP configurado para a interface `tun` pertence à mesma rede do destino do `ping`, permitindo assim o encaminhamento adequado do tráfego.



```
root@0a09040545f0:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
```

Figura 4: `Ping` para o endereço `192.168.53.1`.

```

IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw

```

Figura 5: Encaminhamento correto do tráfego pela interface.

Por outro lado, ao realizar um `ping` para um endereço fora da rede configurada, a interface não consegue responder. Esse comportamento é observado, por exemplo, quando o destino é o endereço `192.168.60.1`, conforme ilustrado nas figuras seguintes.

```

root@0a09040545f0:/# ping 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=0.146 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.104 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=0.109 ms
64 bytes from 192.168.60.1: icmp_seq=4 ttl=64 time=0.110 ms
64 bytes from 192.168.60.1: icmp_seq=5 ttl=64 time=0.117 ms
64 bytes from 192.168.60.1: icmp_seq=6 ttl=64 time=0.092 ms
64 bytes from 192.168.60.1: icmp_seq=7 ttl=64 time=0.102 ms
64 bytes from 192.168.60.1: icmp_seq=8 ttl=64 time=0.118 ms
64 bytes from 192.168.60.1: icmp_seq=9 ttl=64 time=0.102 ms

```

Figura 6: Ping para o endereço `192.168.60.1`.

```

root@0a09040545f0:/volumes# python3 tun.py
Interface Name: tun0
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding

```

Figura 7: A interface não responde a tráfego fora da sua rede.

Questão 2.d

Partindo do código fornecido, foi efetuada uma modificação para que o programa aguardasse especificamente pela recepção de pacotes do tipo ICMP. Após esta alteração, o comportamento observado foi o esperado, como se pode verificar na Figura 8.

```
root@b13bcde1cf11:/# tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:17:59.261431 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 1, length 64
21:17:59.264001 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 1, length 64
21:18:00.263722 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 2, length 64
21:18:00.266122 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 2, length 64
21:18:01.265850 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 3, length 64
21:18:01.267863 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 3, length 64
21:18:02.267183 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 4, length 64
21:18:02.269712 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 4, length 64
21:18:03.269427 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 5, length 64
21:18:03.271216 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 5, length 64
21:18:04.271951 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 6, length 64
21:18:04.273717 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 6, length 64
21:18:05.274074 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 7, length 64
21:18:05.276281 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 7, length 64
21:18:06.276018 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 8, length 64
21:18:06.278140 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 8, length 64
21:18:07.277835 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 9, length 64
21:18:07.279917 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 9, length 64
21:18:08.279194 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 10, length 64
21:18:08.281137 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 10, length 64
21:18:09.280851 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 11, length 64
21:18:09.283296 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 11, length 64
```

Figura 8: Fluxo na interface tun

Por outro lado, quando não é enviado um pacote ICMP e se escreve manualmente na interface, por exemplo:

```
os.write(tun, b'Hello from the other side!')
```

não ocorre qualquer saída, dado que a interface apenas aceita pacotes IP válidos. A Figura 9 ilustra este comportamento.

```
root@b13bcde1cf11:/# tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:32:43.517107 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 4, length 64
21:32:44.541266 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 5, length 64
21:32:45.565245 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 6, length 64
21:32:46.589206 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 7, length 64
21:32:47.613253 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 8, length 64
21:32:48.637320 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 9, length 64
```

Figura 9: Ausência de resposta quando não é recebido um pacote IP.